CSS37W`1 Software Engineering I

Step-by-Step Guide: Setting Up a GitHub Repository for the Project

1. Creating the GitHub Repository

1. One team member (ideally the **Project Manager** or **Lead/Team Developer**) should:
    - Go to GitHub
    - Click on **"New Repository"**
    - Enter a **repository name** (e.g., team-name-project)
    - Set visibility to **Private/Public** (It is to you whether it's public or private)
    - **Check** the box for **"Initialize this repository with a README"**
    - Click **"Create Repository"**

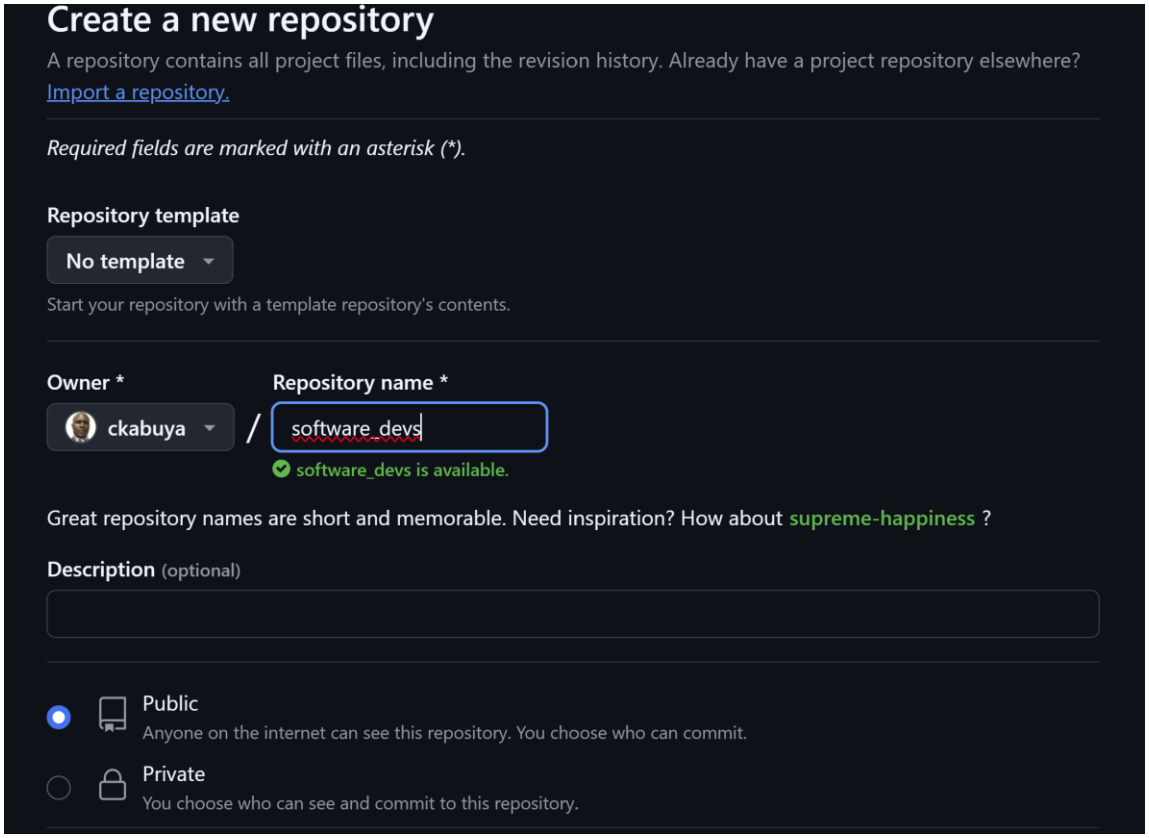

Figure 1 Creating a Repository

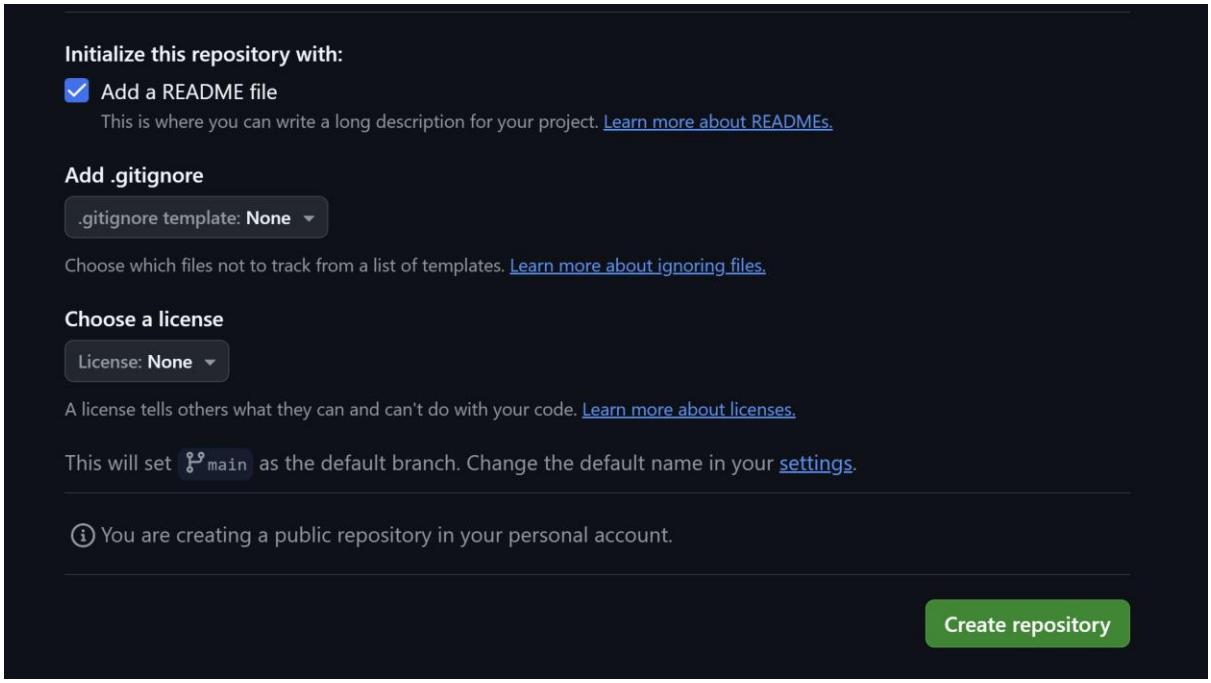

Figure 2 Settings for README file

2. After creating the repository:
    - Click **Settings** > **Manage Access** > **Invite Collaborators**

Code | Issues | Pull requests | Discussions | Actions | Projects | Wiki | Security | Insights

General

Access
- Collaborators
- Moderation options

Code and automation
- Branches
- Tags
- Rules
- Actions
- Webhooks
- Environments
- Codespaces
- Pages

Security
- Code security
- Deploy keys
- Secrets and variables

Integrations
- GitHub Apps
- Email notifications

General

Repository name

software_devs | Rename

☐ **Template repository**
Template repositories let users generate new repositories with the same directory structure and files. Learn more about template repositories.

☐ **Require contributors to sign off on web-based commits**
Enabling this setting will require contributors to sign off on commits made through GitHub's web interface. Signing off is a way for contributors to affirm that their commit complies with the repository's terms, commonly the Developer Certificate of Origin (DCO). Learn more about signing off on commits.

Default branch

The default branch is considered the "base" branch in your repository, against which all pull requests and code commits are automatically made, unless you specify a different branch.

main

Social preview

Upload an image to customize your repository's social media preview.

Images should be at least 640×320px (1280×640px for best display).
Download template

Edit

Features

☑ **Wikis**
Wikis host documentation for your repository.

☑ **Restrict editing to collaborators only**
Public wikis will still be readable by everyone.

☑ **Issues**
Issues integrate lightweight task tracking into your repository. Keep projects on track with issue labels and milestones, and reference them in commit messages.

Get organized with issue templates
Give contributors issue templates that help you cut through the noise and help them push your project forward.
Set up templates

☐ **Sponsorships**
Sponsorships help your community know how to financially support this repository.

Display a "Sponsor" button
Add links to GitHub Sponsors or third-party methods your repository accepts for financial contributions to your project.
Set up sponsor button

☑ **Preserve this repository**
Include this code in the GitHub Archive Program.

☑ **Discussions**
Discussions is the space for your community to have conversations, ask questions and post answers without opening issues.

Get started with Discussions
Engage your community by having discussions right in your repository, where your community already lives
Set up discussions

☑ **Projects**
Projects on GitHub are created at the repository owner's level (organization or user) and can be linked to a repository's Projects tab. Projects are suitable for cross-repository development efforts such as feature work, complex product roadmaps or even Issue triage.

Figure 3 Project Settings

- Add all **team members and the instructor** by entering their GitHub usernames.

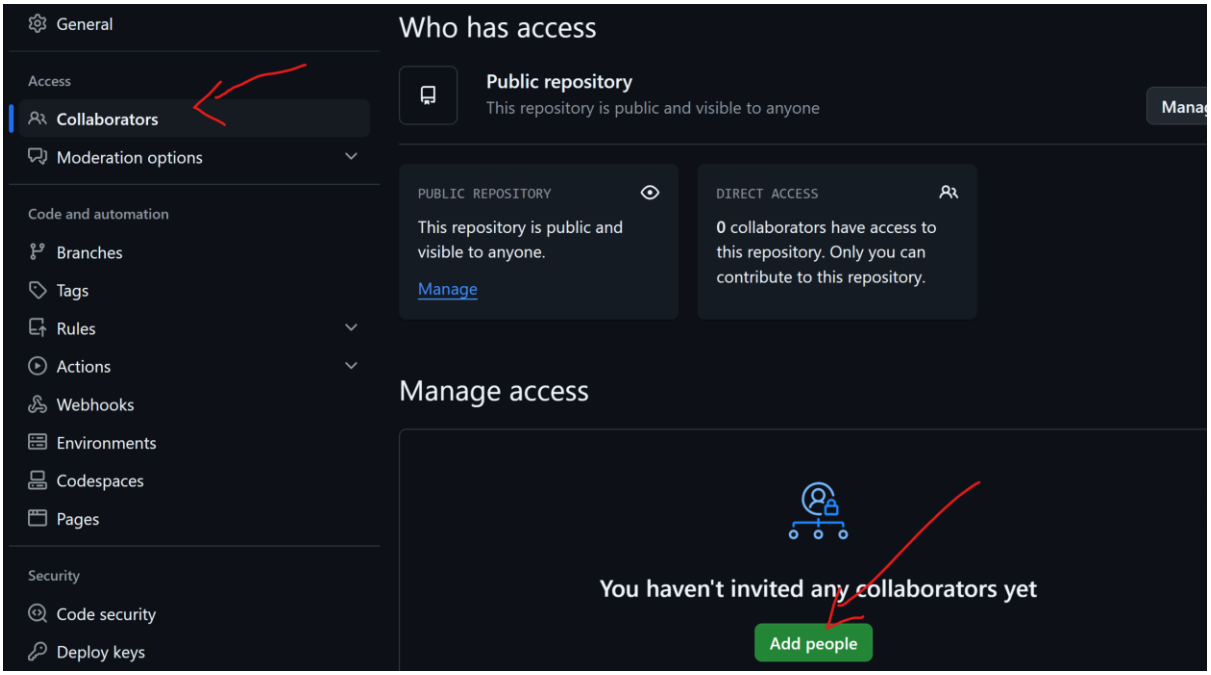

Figure 4 Add collaborators to the project. Please note that you can only add those already registered on GitHub.

2. Initializing the Project Structure

Each repository should have:

- A **README.md** file (which explains the project, team members, and objectives).
- A **.gitignore** file (to exclude unnecessary files like logs, compiled code, or environment settings).
- A **LICENSE** file (optional but useful for open-source projects).

Example README.md File:

```
# Team Name - Project Name

## Project Overview
A brief description of the project and its goals.

## Team Members
- [Name 1] (Role)
- [Name 2] (Role)
- [Name 3] (Role)
- [Name 4] (Role)
- [Name 5] (Role)

## Features
- Feature 1
- Feature 2
- Feature 3

## Technologies Used
- Language/Framework 1
- Database 1
```

```
- Deployment Platform (e.g., AWS, Azure, DigitalOcean)

## Setup Instructions
1. Clone the repo: `git clone https://github.com/team-name/project.git`
2. Install dependencies: `npm install` (or the equivalent)
3. Run the project: `npm start` (or the equivalent)
```

3. Implementing Branching and Merging Strategies

To ensure smooth collaboration, teams should:

- Use **main (or master)** as the stable branch.
- Create **feature branches** for new tasks.
- Use **pull requests (PRs)** for merging changes after review.

Example Workflow:

1. **Create a feature branch**:
2. git checkout -b feature-new-functionality
3. **Work on the feature** and commit changes:
4. git add .
5. git commit -m "Implemented new feature"
6. **Push the branch to GitHub**:
7. git push origin feature-new-functionality
8. **Open a Pull Request** on GitHub:
    - Go to the repository on GitHub.
    - Click **Pull Requests > New Pull Request**.
    - Select **feature-new-functionality** to merge into **main**.
    - Add a description and click **Create Pull Request**.
9. **Team reviews the PR** and once approved:
10. git checkout main
11. git merge feature-new-functionality
12. git push origin main

4. Using .gitignore

To prevent committing unnecessary files, each team should include a .gitignore file based on their technology stack.

Example .gitignore for a Python Project:

```
# Byte-compiled / package files
```

```
__pycache__/
*.py[cod]
*.so
*.log

# Virtual environment
venv/
env/

# IDE config
.vscode/
.idea/
```

Example .gitignore for a Node.js Project:

```
# Logs
logs
*.log
npm-debug.log*

# Dependencies
node_modules/
dist/

# Environment files
.env
```

5. Managing Issues and Pull Requests

- Teams should use **GitHub Issues** to track bugs and tasks.
- Use **Labels** (e.g., bug, enhancement, documentation) to categorize tasks.
- Assign **Issues to Team Members** to keep track of responsibilities.

Example Workflow:

1. **Create an Issue** for each task.
2. Assign it to a team member.
3. Close the issue once the feature is merged via a Pull Request.

Instructor's Role

- Ensure all **teams have created repositories** and added the instructor.
- Check if **GitHub workflows (branches, pull requests, issues)** are being followed.
- Provide **feedback** on the repository structure and collaboration.

Final Summary

To **successfully manage their GitHub repository**, each team must:

- **Set up a repository and invite members**
- **Follow a branching strategy (main + feature branches)**
- **Use GitHub Issues and Pull Requests for task tracking**
- **Include proper documentation (README, .gitignore)**